CLEF SPORT

Base de Datos para Gestion Empresarial

Proyecto Integrador — Corte 2

Estudiante	David Salazar Lasso
Asignatura	Bases de Datos II
Docente	Helen Marcela Bustamante
Institucion	Inst. Univ. Antonio Jose Camacho
Fecha	Mayo 2026

2.1 — Indices y Rendimiento

5 consultas criticas del negocio optimizadas con indices especificos para clef_sport.

Consultas e indices creados

Consulta	Tabla principal	Tipo de indice	Nombre del indice
Historial por cedula	cliente	Simple	idx_cliente_cedula
Catalogo marca + categoria	producto	Compuesto	idx_prod_marca_cat
Ventas por rango de fechas	venta	Simple (rango)	idx_venta_fecha
Productos con stock bajo	producto	Simple (rango)	idx_prod_stock
Ranking de productos vendidos	detalle_venta	Covering	idx_dv_prod_covering

Scripts de creacion

```
-- Indice 1: Simple — busqueda por cedula (C1)
CREATE INDEX idx_cliente_cedula    ON cliente(cedula);

-- Indice 2: Compuesto — filtro catalogo marca + categoria (C2)
CREATE INDEX idx_prod_marca_cat    ON producto(id_marca, id_categoria);

-- Indice 3: Simple rango — reportes por fecha (C3)
CREATE INDEX idx_venta_fecha       ON venta(fecha);

-- Indice 4: Simple rango — alerta stock bajo (C4)
CREATE INDEX idx_prod_stock        ON producto(stock);

-- Indice 5: Covering — ranking de ventas sin tocar la tabla (C5)
CREATE INDEX idx_dv_prod_covering  ON detalle_venta(id_producto, cantidad, subtotal);
```

Comparativa antes / despues

Consulta	Antes (sin indice)	Despues (con indice)
C1 — cedula	type=ALL | rows=50	type=ref | rows=1
C2 — marca+cat	type=ALL | rows=50	type=ref | rows~8
C3 — fecha	type=ALL | rows=50	type=range | rows~10
C4 — stock<5	type=ALL | rows=50	type=range | rows~3
C5 — covering	type=ALL + filesort	type=index | Using index

2.2 — Logica de Negocio Avanzada

5 procedimientos almacenados, 3 funciones y 3 triggers sobre clef_sport.

Procedimientos Almacenados

SP	Que hace	Prueba
sp_registrar_venta	Valida stock, registra venta y descuenta inventario	CALL sp_registrar_venta(1, 2, CURDATE(), 'Efectivo', 1, 1);
sp_registrar_compra	Registra compra a proveedor y suma stock	CALL sp_registrar_compra(1, 1, CURDATE(), 2, 10, 380000);
sp_actualizar_precio_categoria	Ajusta precios por categoria con margen minimo 15%	CALL sp_actualizar_precio_categoria(1, 5.00);
sp_anular_venta	Anula venta (estado) y restaura el stock	CALL sp_anular_venta(50);
sp_reporte_ventas_periodo	Reporte por dia o mes segun amplitud del rango	CALL sp_reporte_ventas_periodo('2025-08-01','2025-11-30');

Funciones

Funcion	Que retorna	Uso en consulta
fn_margen_producto(venta, costo)	Margen de ganancia en %	SELECT nombre, fn_margen_producto(precio_venta, precio_costo) FROM producto
fn_estado_stock(stock)	AGOTADO / CRITICO / BAJO / NORMAL / SUFICIENTE	SELECT nombre, fn_estado_stock(stock) FROM producto ORDER BY stock
fn_total_vendido_cliente(id)	Total acumulado de compras del cliente	SELECT nombre, fn_total_vendido_cliente(id_cliente) FROM cliente

Triggers

Trigger	Tipo	Que hace
trg_auditoria_stock	AFTER UPDATE en producto	Registra en auditoria_stock cada cambio de stock con valor anterior, nuevo y fecha
trg_validar_stock	BEFORE INSERT en detalle_venta	Verifica stock suficiente antes de insertar; lanza error 45000 si no alcanza
trg_total_venta	AFTER INSERT en detalle_venta	Recalcula y actualiza automaticamente el campo total en la tabla venta

Manejo de errores en SP criticos

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    RESIGNAL;   -- relanza el error al llamador
END;

-- Uso: todos los SP con transaccion llevan este handler
-- Si cualquier instruccion falla, el ROLLBACK es automatico
```

2.3 — Transacciones y Concurrencia

Nivel de aislamiento, 3 transacciones probadas y simulacion de deadlock.

Nivel de aislamiento

Nivel: REPEATABLE READ (por defecto en InnoDB/MySQL)

Por que: Clef Sport vende ediciones limitadas con stock reducido. REPEATABLE READ garantiza que dos vendedores que consulten el mismo producto en paralelo vean el mismo stock durante toda su transaccion, evitando double-selling.

READ COMMITTED descartado: permite lecturas no repetibles que podrian vender mas unidades de las existentes entre la consulta y el UPDATE.

Complemento: SELECT ... FOR UPDATE en sp_registrar_venta bloquea la fila del producto durante el ciclo completo de la transaccion.

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT @@transaction_isolation;    -- 'REPEATABLE-READ'
```

Transaccion 1 — Venta exitosa

Ciclo completo: bloqueo del producto, venta, descuento de stock, COMMIT.

```
START TRANSACTION;
  SELECT id_producto, stock FROM producto WHERE id_producto = 1 FOR UPDATE;
  INSERT INTO venta (id_cliente, id_usuario, fecha, total, metodo_pago, estado)
    VALUES (3, 2, CURDATE(), 950000, 'Transferencia', 'Completada');
  INSERT INTO detalle_venta (id_venta, id_producto, cantidad, precio_unitario, subtotal)
    VALUES (LAST_INSERT_ID(), 1, 1, 950000, 950000);
  UPDATE producto SET stock = stock - 1 WHERE id_producto = 1;
COMMIT;
-- Stock reducido en 1. trg_auditoria_stock registra el cambio automaticamente.
```

Transaccion 2 — Fallo y ROLLBACK (consistencia)

Demuestra que sin COMMIT ningun cambio persiste, aunque el INSERT ya haya ocurrido.

```
START TRANSACTION;
  INSERT INTO venta (id_cliente, id_usuario, fecha, total, metodo_pago, estado)
    VALUES (7, 3, CURDATE(), 1500000, 'Efectivo', 'Completada');
  -- trg_validar_stock rechaza 9999 unidades (stock real = 30)
  -- INSERT INTO detalle_venta (..., 9999, ...)  --> ERROR 45000
ROLLBACK;
-- La venta insertada NO existe en la BD.
```

```
-- El stock del producto queda intacto (Atomicidad demostrada).
```

Transaccion 3 — SAVEPOINT (rollback parcial)

Registra una compra conservando el encabezado aunque el detalle falle.

```
START TRANSACTION;
  INSERT INTO compra (...) VALUES (1, 1, CURDATE(), 12000000, 'Recibida');
  SAVEPOINT sp_encabezado_ok;
  INSERT INTO detalle_compra (...) VALUES (..., 1, 10, 600000);  -- OK
  UPDATE producto SET stock = stock + 10 WHERE id_producto = 1;
  SAVEPOINT sp_primer_detalle_ok;
  -- Segundo detalle con FK invalida → error
  ROLLBACK TO SAVEPOINT sp_primer_detalle_ok;
COMMIT;  -- compra + primer detalle quedan; segundo no
```

Simulacion de Deadlock

Dos vendedores intentan bloquear los mismos productos en orden inverso.

Paso	Sesion A	Sesion B
1	START TRANSACTION; SELECT ... id=1 FOR UPDATE	(espera)
2	(espera)	START TRANSACTION; SELECT ... id=2 FOR UPDATE
3	UPDATE ... id=2 → espera lock de B	—
4	—	UPDATE ... id=1 → espera lock de A → DEADLOCK
Resultado	Continua y puede hacer COMMIT	ERROR 1213: InnoDB aborta automaticamente

Diagnostico y prevencion

```
SHOW ENGINE INNODB STATUS\G                    -- ver ultimo deadlock
SELECT * FROM information_schema.INNODB_TRX;   -- transacciones activas
SELECT * FROM performance_schema.data_locks;   -- locks activos
```

Prevencion en Clef Sport:

- Acceder siempre a los productos en el mismo orden (ORDER BY id_producto ASC) dentro de sp_registrar_venta.
- Mantener transacciones cortas: bloquear → operar → confirmar sin logica intermedia.
- Indices en columnas usadas con FOR UPDATE para minimizar el rango de filas bloqueadas.